Processingをはじめよう

第9章

オブジェクト

課題　解答

Kadai_09_01（必ず提出）

- JitterBugクラスを改良する
  - グレイスケールで色を付ける（0～255）
    - フィールド追加　gray（整数型）
    - コンストラクタ
      - 色を指定するための引数追加　tempGray（整数型）
      - コンストラクタで色の値を代入するように修正
    - メソッド display()の修正
      - 円を描く前にgrayを使って塗る色を指定する

変更前 Example 9-1

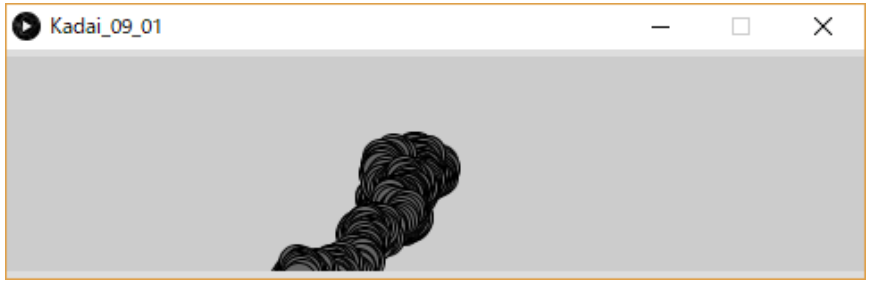
```
JitterBug bug;

void setup() {
  size(480, 120);
  bug = new JitterBug(width/2,
                          height/2, 20);
}

void draw() {
  bug.move();
  bug.display();
}
```

```
// 新規タブを追加して,JitterBugクラスを書く
class JitterBug {
  float x;
  float y;
  int diameter;
  float speed = 2.5;

  JitterBug(float tempX, float tempY,
                        int tempDiameter) {
    x = tempX;
    y = tempY;
    diameter = tempDiameter;
  }

  void move() {
    x += random(-speed, speed);
    y += random(-speed, speed);
  }

  void display() {
    ellipse(x, y, diameter, diameter);
  }
}
```

変更後 Kadai_09_01

```
JitterBug bug;

void setup() {
  size(480, 120);
  bug = new JitterBug(width/2,
                      height/2, 20, 102);
}

void draw() {
  bug.move();
  bug.display();
}
```

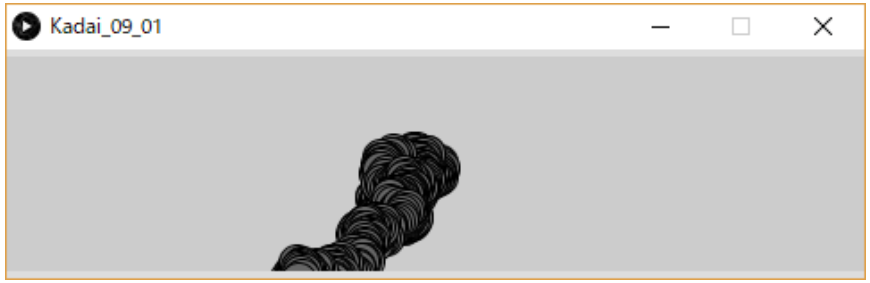
修正
灰色を指定




```
class JitterBug {
  float x;
  float y;
  int diameter;
  float speed = 2.5;
  フィールドの追加

  JitterBug(float tempX, float tempY,
          int tempDiameter, 引数追加 ) {
    x = tempX;
    y = tempY;
    diameter = tempDiameter;
    フィールドへの代入
  }
  void move() {
    x += random(-speed, speed);
    y += random(-speed, speed);
  }
  void display() {
    図形内部をgrayで塗る
    ellipse(x, y, diameter, diameter);
  }
}
```

Kadai_09_01 解答例

```
JitterBug bug;

void setup() {
  size(480, 120);
  bug = new JitterBug(width/2,
                          height/2, 20, 102);
}

void draw() {
  bug.move();
  bug.display();
}
```




```
class JitterBug {
  float x;
  float y;
  int diameter;
  float speed = 2.5;
  int gray;

  JitterBug(float tempX, float tempY,
          int tempDiameter, int tempGray) {
    x = tempX;
    y = tempY;
    diameter = tempDiameter;
    gray = tempGray;
  }
  void move() {
    x += random(-speed, speed);
    y += random(-speed, speed);
  }
  void display() {
    fill(gray);
    ellipse(x, y, diameter, diameter);
  }
}
```

Kadai_09_02（必ず提出）

- 学生 Student クラスを作る
  - フィールド
    - 文字列型　名前　　name
    - 文字列型　学籍番号　　studentNumber
  - コンストラクタで
    名前と学籍番号のフィールドに初期値を代入
    - 名前はローマ字表記で代入　例 青森太郎の場合　"Aomori Taro"
    - 学籍番号は"si28???"(?は自分の数字)の文字列で代入
  - メソッド
    - 名前と学籍番号をコンソールに表示する show()
      名前と学籍番号の間には、", "を入れる(コンマとスペース)

Kadai_09_02

Studentクラスのオブジェクト変数を宣言

オブジェクト名は
苗字(小文字表記)

```
void setup(){
```

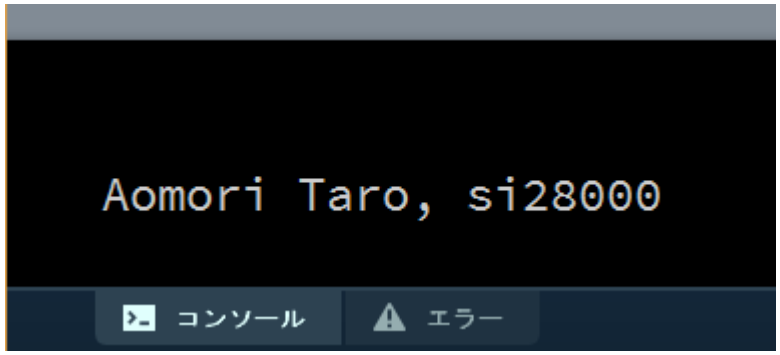
new演算子を使ってオブジェクトを生成

```
}

void draw(){
```

オブジェクト変数の名前と学籍番号を表示

```
  noLoop();            //繰り返しを一時停止する
}
```

実行結果
名前　　"Aomori Taro"
学籍番号　"si28000"




```
class Student {
```

フィールド　名前

フィールド　学籍番号

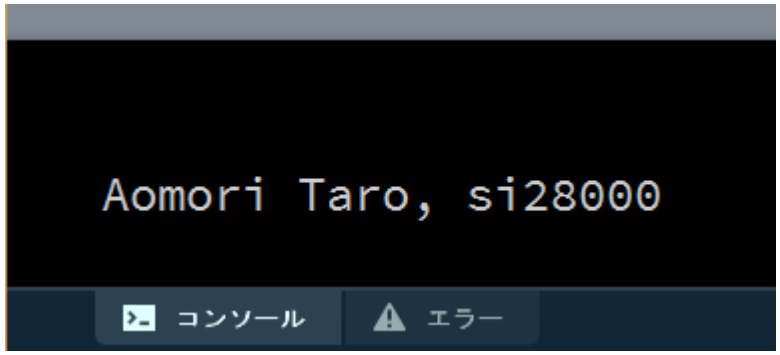
コンストラクタ

メソッド　名前と学籍番号を表示

```
}
```

Kadai_09_02　解答例

```
Student aomori;

void setup(){
  aomori = new Student("Aomori Taro",
                                  "si28000");
}

void draw(){
  aomori.show();
  noLoop();            //繰り返しを一時停止する
}
```

```
class Student {

  String name;
  String studentNumber;

  Student(String name_, String number_){
    name = name_;
    studentNumber = number_;
  }

  void show(){
    println(name + ", " + studentNumber);
  }

}
```

実行結果
名前　　"Aomori Taro"
学籍番号　"si28000"